

Learning with decision trees

Lee-Ad Gottlieb

Decision trees

- A very popular approach is using decision trees for machine **learning**.
- Before discussing the learning algorithms, we'll survey the use of decision trees for **information theory**.

20 questions

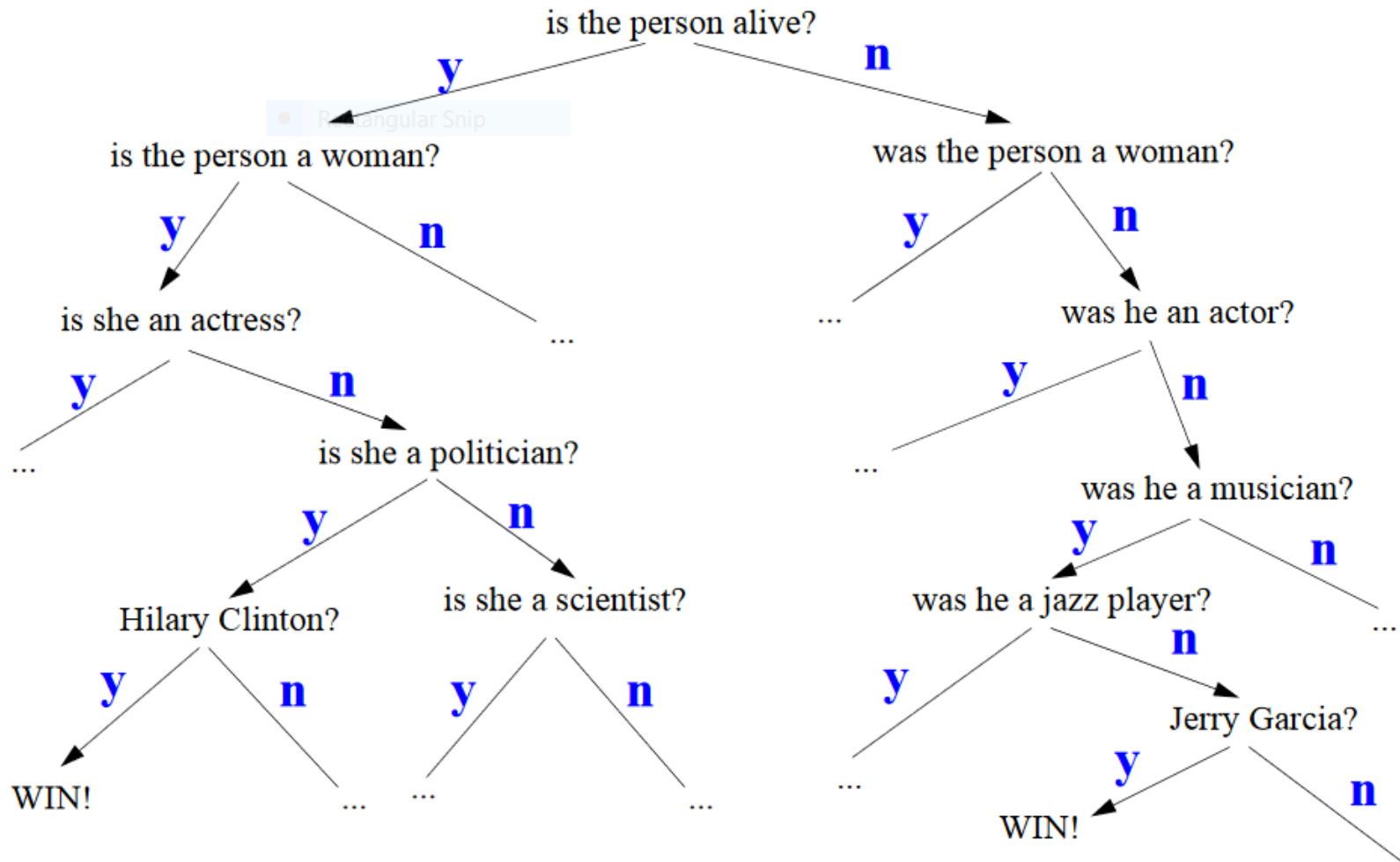
- I'm thinking of a person... can you guess who?
 - You can only ask binary (yes-no) questions
 - At most 20 questions...

20 questions

- I'm thinking of a person... can you guess who?
 - You can only ask binary (yes-no) questions
 - At most 20 questions...

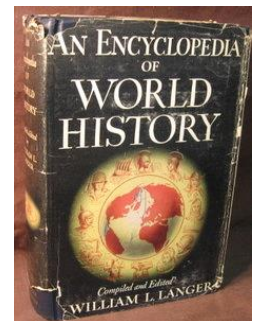


A (partial) 20-questions strategy tree



20 questions

- A binary tree with 20 levels has
 - $2^{20} = 1,048,576$ leaves.
 - That's much larger than the number of people I know of.
- So you should be able to win the game every time
 - At least if you have a good splitting strategy
 - For example, can use binary search dictionary (or encyclopedia) of people's names



A different game

- What if my goal is to ask the smallest number of questions on average?
 - Minimize expected number of guesses: $\Pr[x_i] * \text{number_of_guesses}[x_i]$
- For example
 - in our game above, suppose some people are much more likely to be chosen than others...

Pr



> Pr



- Up next!
 - Long detour into information theory.

Information theory

- Familiar compression schemes:
 - zip, 7z, rar, jpeg.
- How compressed can a file get?
 - The field of information theory studies this.
- Let an **item** be (for example) a letter in a long text...

Information theory

- Founding theorem of Information Theory:
- Shannon (1948): If item i has frequency w_i then the optimal compression of a message of length m is of size (bits)

$$m \sum_i w_i \left[\log_2 \left(\frac{1}{w_i} \right) \right] = -m \sum_i w_i [\log_2 w_i]$$


Entropy function

- Example:
 - document of length $m=50$
 - ‘a’ appears 20 times, ‘b’ appears 30 times.
 - Possible compression: $50 \left(\frac{20}{50} \log_2 \left(\frac{50}{20} \right) + \frac{30}{50} \log_2 \left(\frac{50}{30} \right) \right) = 48.54$ bits
- Fine print: assumes
 - Each item is encoded in a finite string of bits.
 - representation of each item cannot change during encoding.

Entropy function

- Measures how random (or balanced) a set is
- Entropy function for item/event e_i
$$\sum_i \Pr[e_i] \log_2 [1/\Pr[e_i]]$$

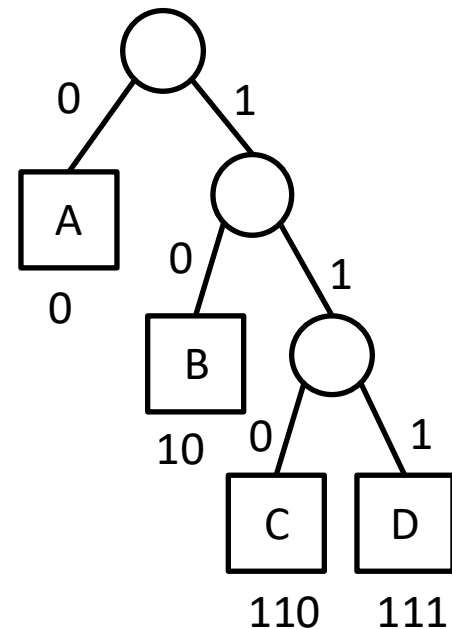
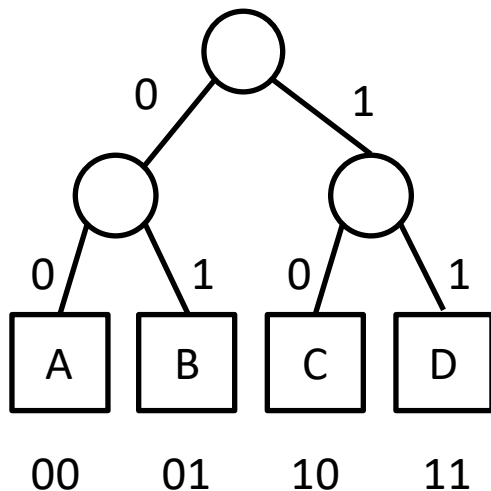


- For example, a fair coin has entropy 1, and a coin that always falls on heads has entropy 0.
- Binary entropy function for item/events e_1, e_2

$$\Pr[e_1] \log \left(1/\Pr[e_1] \right) + \Pr[e_2] \log \left(1/\Pr[e_2] \right) = \\ \Pr[e_1] \log \left(1/\Pr[e_1] \right) + (1 - \Pr[e_1]) \log \left(1/1 - \Pr[e_1] \right)$$

Information theory

- Shannon's theorem is non-constructive
 - He didn't give a code that met his bounds
- Huffman (1952): bottom-up construction realizes optimal bound
 - Give shorter codes to more frequently occurring items
 - First example: BAABAC = 010000010010
 - Second example: BAABAC = 1000100110

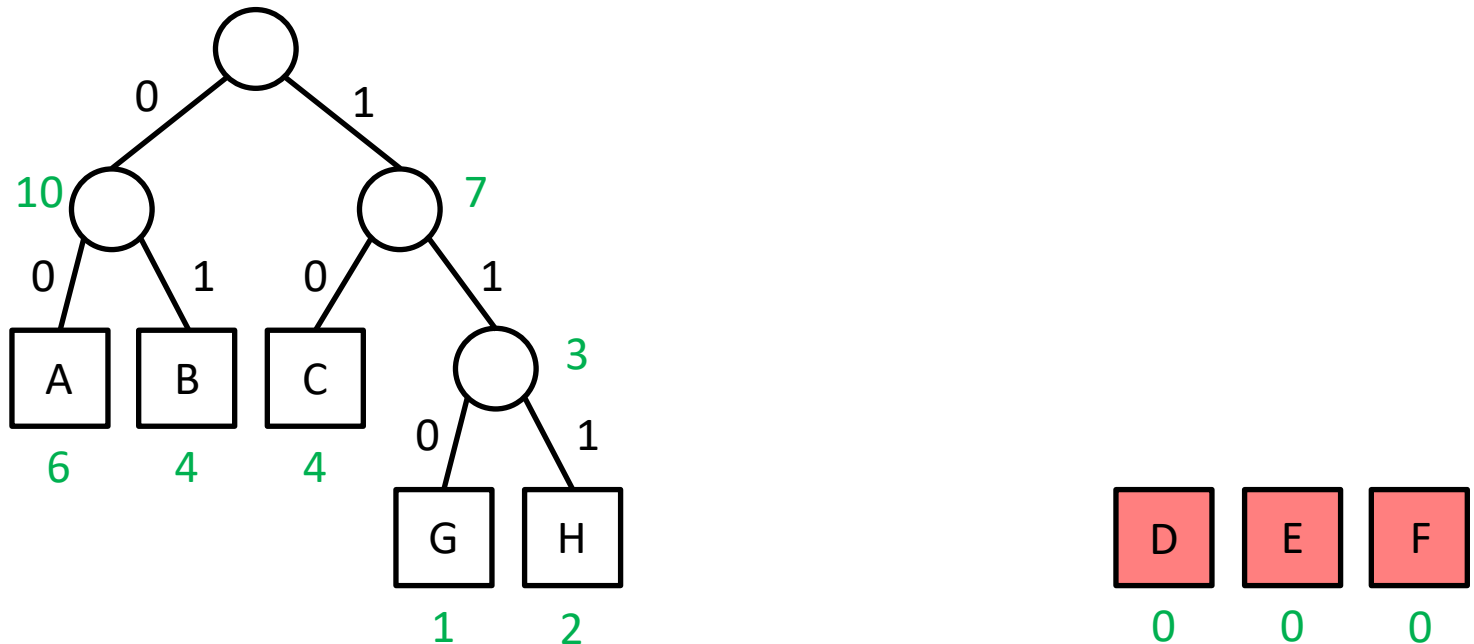


Information theory

- Items: A,B,C,D,E,F,G,H
- Message: AAAAABBAHHBCBGCCC
- Could construct a balanced tree for coding
 - 8 items
 - Each encoded into 3 bits
 - Message size: $3 * 17 = 51$ bits
- Can do better if we look at frequencies,
 - A:6, B:4, C:4, D:0, E:0, F:0, G:1, H:2

Huffman coding

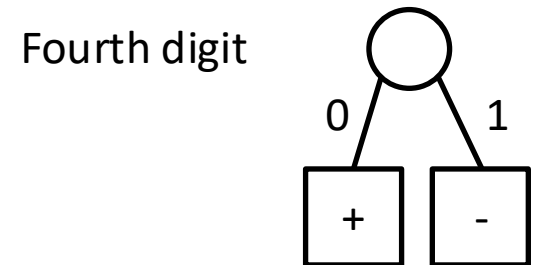
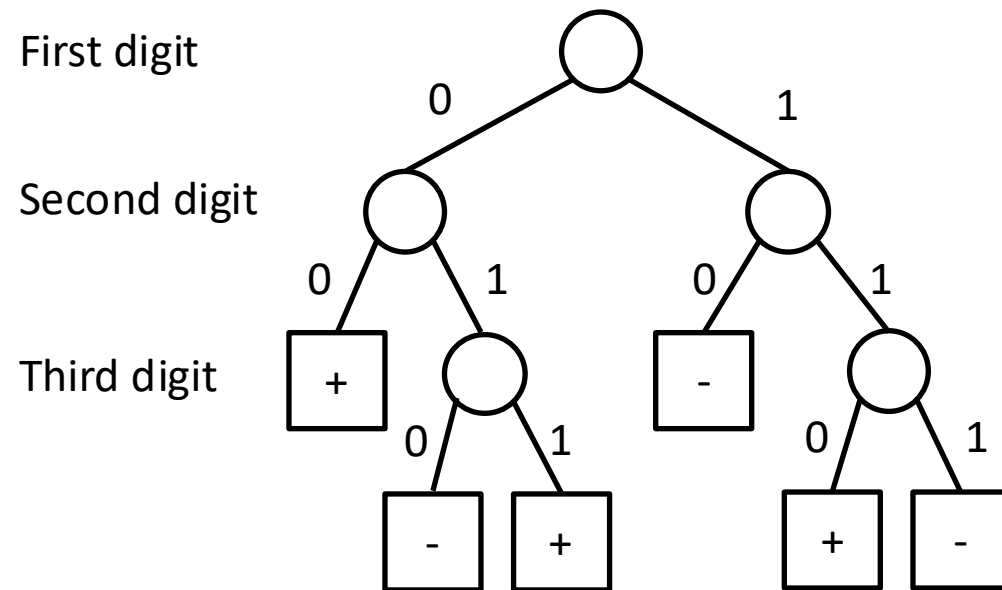
- Build tree bottom-up, combining lightest nodes (Frequencies A:6, B:4, C:4, D:0, E:0, F:0, G:1, H:2)
- AAAAABBAHHBCBGCCC encodes to
- 00000000000010100111111011001110101010 – 37 bits



Learning with decision trees

- We want to learn with decision trees.
 - Example: sample of binary vectors
- Split rule: value in each dimension

0000	+
0010	+
0101	-
0110	+
1001	-
1011	-
1100	+
1111	-



Learning with decision trees

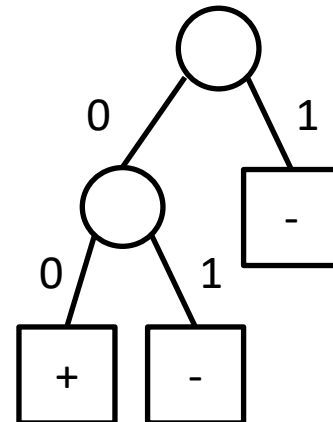
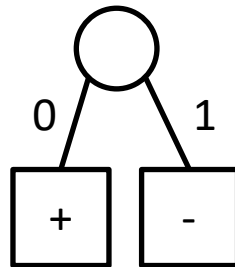
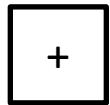
- What's the VC-dimension of decision trees on binary d -dimensional vectors ?
 - $\mathbf{T}(k)$ – family of decision trees with k nodes
 - At each node, choose dimension to test, or mark it as a leaf in $\{+,-\}$ – $d+2$ possible decisions
 - Size of $\mathbf{T}(k) \leq (d+2)^k$ trees
 - $\text{VC-dim}(\mathbf{T}(k)) = O(k \log d)$ (using Sauer's lemma)

Learning with decision trees

- So we want small k .
- **Problem:** find decision tree which is consistent with the data and has the smallest number of nodes.
 - NP-hard.
- Heuristics used instead.
 - ID3
 - Pruning

Random forest

- ID3 algorithm. Top-down:
 - Start with root as a + leaf.
 - Function **Cost** measures quality of solution
 - Split leaf with rule that minimizes **Cost**
 - Repeat until consistent

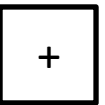


Random forest

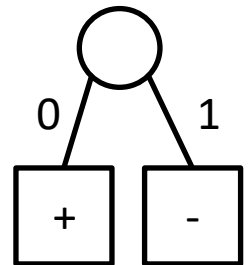
- What's a good **Cost** function?
 - Minimize sum of entropy of leaves
 - Given a set S , choose a rule partitioning S into $A, B \subseteq S$ minimizing

$$\sum_{e_i \in A} \Pr[e_i] \log_2 \left[\frac{1}{\Pr[e_i]} \right] + \sum_{e_i \in B} \Pr[e_i] \log_2 \left[\frac{1}{\Pr[e_i]} \right]$$

- Recall: balanced set has entropy 1. The more unbalanced the set, the smaller the entropy.

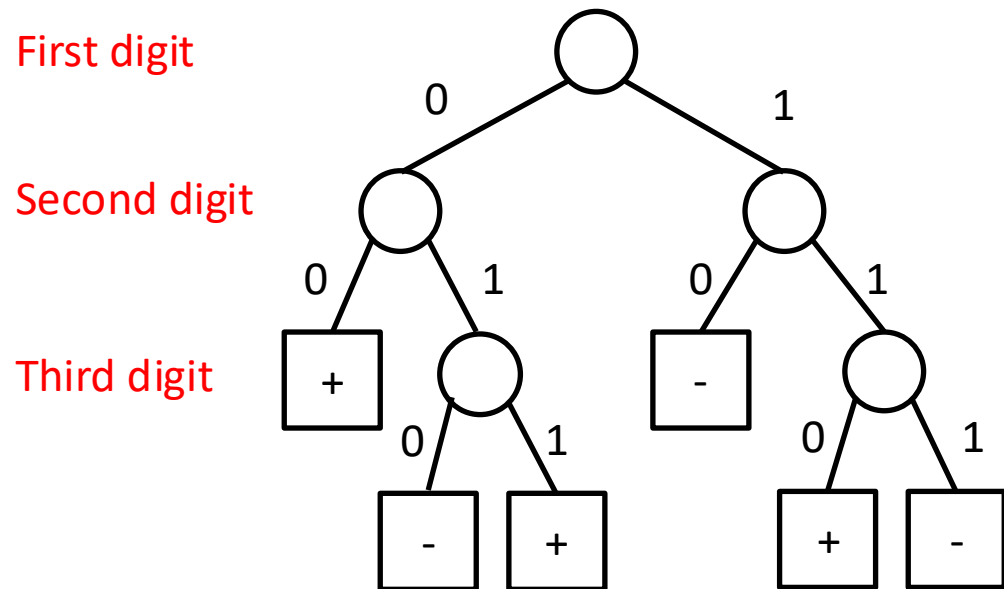


- But no actual guarantees on tree size



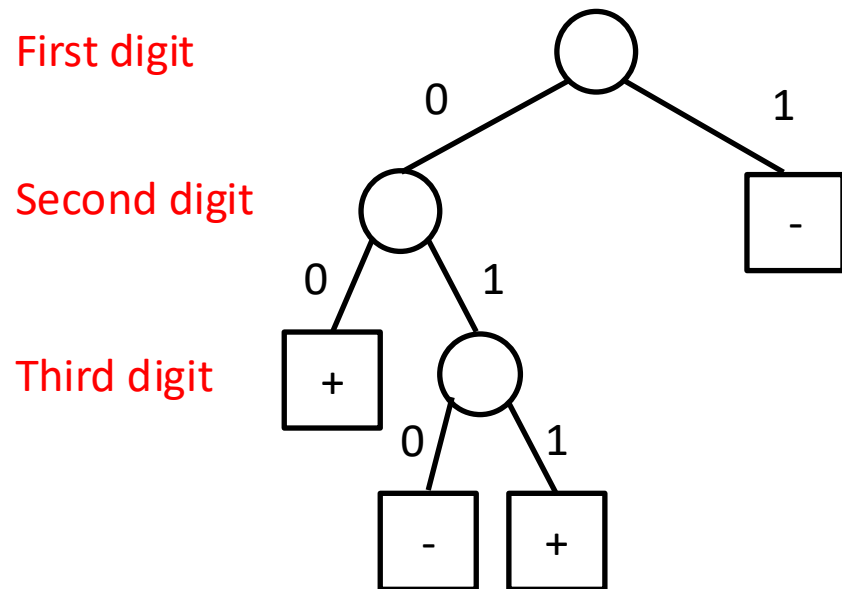
Pruning

- Another way to achieve small tree - **inconsistent**
 - Can do it after brute-force or ID3
- Remove nodes starting at the bottom. Options:
 - **Do nothing**
 - Make internal node into leaf in $\{0,1\}$
 - Replace with subtree



Pruning

- Another way to achieve small tree - **inconsistent**
 - Can do it after brute-force or ID3
- Remove nodes starting at the bottom. Options:
 - Do nothing
 - **Make internal node into leaf in {0,1}**
 - Replace with subtree



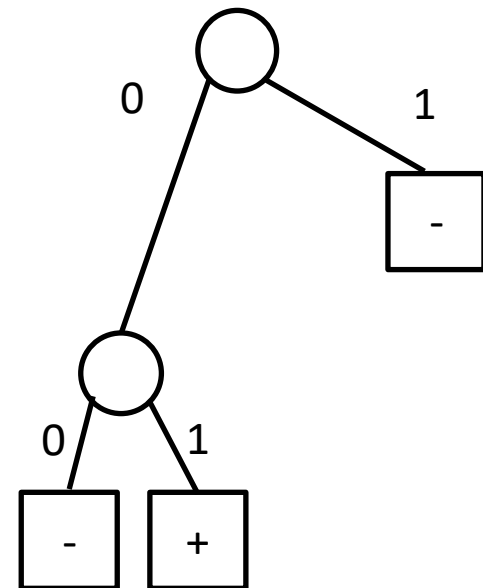
Pruning

- Another way to achieve small tree - **inconsistent**
 - Can do it after brute-force or ID3
- Remove nodes starting at the bottom. Options:
 - Do nothing
 - **Make internal node into leaf in {0,1}**
 - Replace with subtree

First digit

Second digit

Third digit



Random forest

- State-of-the-art
- Compute many random decision trees
 - ID3 with pruning, for example
- Take majority decision

